

Leveraging Quantum Computing for Optimal Data Allocation in Distributed Systems

Immanuel Trummer
Cornell University
Ithaca, New York, USA
itrummer@cornell.edu

Abstract

We show how to use quantum computing to optimize data allocation in distributed systems. Our problem model considers storage constraints (modeling limitations in storage capacity on each node), k -safety constraints (ensuring that a user-defined number of copies of each data partition is distributed in the system), and performance (calculating overheads due to the need for sending data for remote accesses over the network). Specifically, our approach leverages quantum annealers, providing high-quality solutions to instances of quadratic unconstrained binary optimization (QUBO) problems quickly. We show how to transform data allocation instances into equivalent QUBO instances. Also, we analyze the number of qubits needed to represent the resulting instances. More precisely, we determine the asymptotic number of required qubits as a function of the dimensions of the input problem. Our analysis takes into account different annealer topologies, ranging from fully connected qubits to sparse connection structures. Finally, we outline future work for this project.

CCS Concepts

• **Information systems** → **Autonomous database administration**; *Parallel and distributed DBMSs*; • **Hardware** → **Quantum computation**.

Keywords

data partitioning, quantum computing

ACM Reference Format:

Immanuel Trummer. 2025. Leveraging Quantum Computing for Optimal Data Allocation in Distributed Systems. In *Workshop on Quantum Computing and Quantum-Inspired Technology for Data-Intensive Systems and Applications (Q-Data '25)*, June 22–27, 2025, Berlin, Germany. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3736393.3736692>

1 Introduction

Distributed systems for data processing store copies of data partitions on multiple nodes for several reasons. First, storing multiple data copies can improve performance. Answering a request from data stored locally is typically more efficient, as it avoids the need

to send data over the network. Second, storing multiple copies is crucial for data recovery in case of faulty nodes. For instance, systems like H-Store [22] support k -safety, i.e., data is stored on at least k nodes and faults of up to $k - 1$ nodes are recoverable. However, node storage capacity is limited. This means we cannot store all the data on each node. Hence, we must choose carefully which data to assign to which nodes.

The aforementioned tension leads to the problem of data allocation (or assignment) in distributed systems [3]. Typically, variants of the data assignment problem are NP-hard. Hence, no known algorithms on classical computers is able to reliably calculate optimal solutions with a polynomial run time. The hardness of the problem, combined with the significant performance impact of its solutions, motivates the search for novel approaches for solving it.

Quantum computing has recently turned from a purely theoretical concept into a computing paradigm for which a growing number of hardware implementations are readily available. Providers such as Amazon, IBM, D-Wave, or Google nowadays offer access to quantum machines via the Cloud, making such devices accessible to broad shares of the population. For problems such as search in unstructured databases, quantum computers achieve asymptotic performance advantages over classical machines [16]. This has motivated a line of recent research that applies quantum computing to solve hard optimization problems that arise in the context of database systems [6, 11, 14, 25, 31, 38, 39, 44]. In this work, we take the first steps toward solving the problem of optimal data assignment in a distributed system on quantum computers.

We focus specifically on quantum annealers. Those machines, discussed in more detail in Section 2, use quantum computing to solve a specific class of optimization problems: QUBO. The goal of QUBO is to optimize a quadratic cost formula that depends on binary variables. To solve any optimization problem on a quantum annealer, we must first transform problem instances from the original format into QUBO instances. We show a transformation that maps data allocation problem instances to QUBO. Our problem model for data allocation considers two types of constraints. First, it considers storage constraints, meaning that the accumulated size of all data assigned to a node must not exceed its storage capacity. Second, it considers k -safety constraints. As implemented in systems like H-Store [22], k -safety ensures that each data partition is available on at least k nodes. Therefore, up to $k - 1$ node failures can be tolerated without losing data.

Our goal is to optimize processing performance under the two aforementioned constraints. Currently, we model processing costs as proportional to the amount of data sent over the network. Each node in the system receives requests from users that require access

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Q-Data '25, Berlin, Germany

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1944-8/25/06
<https://doi.org/10.1145/3736393.3736692>

to specific data partitions. If requested partitions are available locally, no data needs to be sent over the network. Otherwise, the system incurs costs for sending data. We assume that the frequency at which specific partitions are required is known for each node, as well as the cost of sending specific partitions. Given that information, we model cost as a function of the assignments, determining which requests can be answered from local data and which requests require communication.

We transform problem instances into a QUBO formula to minimize. The variables of this QUBO formula represent assignments from data copies to specific nodes. The formula takes on high values if storage or k-safety constraints are violated. As the goal is to minimize the value of the formula, this discourages value assignments to problem variables that represent invalid assignments. If no constraints are violated, the value of the formula is equivalent to processing costs. Hence, a solution satisfying all constraints while minimizing processing overheads minimizes the QUBO formula and represents an optimal solution to the original data allocation problem at the same time.

The number of qubits is one of the primary limitations of current annealer machines (ranging up to only a few thousand qubits). Therefore, it is crucial to analyze QUBO transformations in terms of the required number of qubits. We analyze the asymptotic number of required qubits as a function of the dimensions of the input problem (data allocation). Specifically, we take into account the number of nodes and data partitions, as well as the range of capacity thresholds (as representing higher capacity values require more qubits). For a topology in which qubits are fully connected, enabling an annealer to model quadratic QUBO terms between arbitrary variables, it is sufficient to represent one problem variable by a single qubit. However, typically, qubits are only connected to a limited number of other qubits. Hence, we might need to represent a problem variable by multiple qubits, thereby making sure that all quadratic terms involving that variable can be represented. We analyze the required number of qubits under different assumptions, ranging from full to sparse connections between qubits (meaning that each qubit is only connected to a constant number of other qubits).

In summary, the original scientific contributions in this paper are the following:

- We model data allocation as an optimization problem and prove that it is NP-hard.
- We propose a transformation from data allocation to equivalent QUBO problems.
- We analyze how the number of required qubits grows as a function of the input problem dimensions.

The remainder of this paper is organized as follows. Section 2 discusses the required background on quantum computing and quantum annealers. Section 4 shows how to leverage quantum computing to solve data allocation instances. Section 5 introduces a problem transformation, transforming instances of data allocation to semantically equivalent QUBO formulas. Section 6 analyzes the hardness of our data allocation variant and analyzes how the number of required qubits depends on the dimensions of the input problem. Finally, Section 8 summarizes results and discusses next steps.

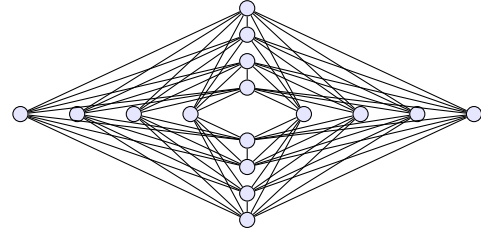


Figure 1: One cell in D-Wave's Zephyr topology, featuring 16 connected qubits.

2 Background on Quantum Computing

Classical computers operate on bits. Consistent with the laws of classical physics, each bit can only be in one of two possible states (1 or 0) at the same time. Quantum mechanics, however, expands over classical physics by allowing superposition of states that would be considered mutually exclusive according to classical physics. Hence, according to quantum physics, a bit (called *qubit* in the context of quantum computing) may be in a superposition of states (1 and 0) that would be considered mutually exclusive, according to classical physics.

Quantum computers exploit opportunities offered by quantum mechanics for computation. For certain problems, e.g., the problem of finding items in an unstructured data set, algorithms executed on quantum computers offer some asymptotic speedups [16], compared to the best possible algorithm on a classical computer. On the other hand, it is currently unclear whether quantum algorithms offer exponential speedups for NP-hard optimization problems [1].

Quantum computing can be implemented in different ways. Currently, multiple physical realizations of quantum computing are available, including the circuit model [50] (which substitutes logical gates of classical computers with specialized gates that operate on quantum states) as well as quantum annealing. We focus on exploiting quantum annealing in this publication (while leaving studies related to the quantum circuit model for future work). Next, we describe quantum annealing as implemented by machines by the Canadian company D-Wave.

A quantum annealer features a matrix of qubits, realized as tiny loops manufactured from the metal Niobium. Qubit states are represented by the direction of power flowing through those loops. In a classical setup, power can only flow into one of the two possible directions, representing a state of 1 or 0. However, at low temperatures, Niobium becomes superconducting, allowing a superposition of states (i.e., power flowing in both directions) that represents a superposition of bit states. This is why the circuits of D-Wave's annealers are cooled down to extremely low temperatures, enabling quantum computation.

Quantum annealers implement adiabatic quantum computing, a process that (in the absence of noise) finds minimum energy states. Starting from a superposition of states, qubits converge to value combinations that minimize the energy level of the system. The function that maps qubit values to energy levels can be configured by placing (positive or negative) weights on single qubits, as well as on pairs of qubits. Weights associated with qubit pairs are incurred if both connected qubits are set to one. In summary, the qubit

hardware allows representing instances of quadratic unconstrained binary optimization (QUBO), i.e., minimizing a sum of linear and quadratic terms depending on binary variables (a more formal definition is given in Section 3).

Current quantum annealers do not feature connections between all qubit pairs. For instance, Figure 1 shows connections between qubits in a single cell of the Zephyr architecture, used by recent models of D-Wave’s quantum annealers. When mapping QUBO formulas to physical annealer hardware, quadratic terms in the QUBO formula must be mapped to connections between qubits. As the number of quadratic terms in which a QUBO variable appears may exceed the number of connections available per qubit, QUBO variables have to be mapped not to single qubits but to connected groups of qubits. This means that QUBO formulas requiring dense connections between problem variables may require a number of qubits to represent that is significantly higher than the number of variables. In Section 6, we discuss the number of qubits required for our mapping approach in more detail for different annealer architectures.

The rapid development of quantum annealers has motivated an ecosystem of corresponding libraries, inspiring in turn the development of classical hardware accelerators for solving similar problem types. Those hardware accelerators are often referred to as *digital annealers* and include devices such as Fujitsu’s Digital Annealer [24, 30, 37], NEC’s vector annealing machine [34], Toshiba’s Simulated Bifurcation Machine [13], Fixstar’s Amplify Annealing Engine [23], and the CMOS annealing machine [48]. As those machines support the same problem format, the approach proposed in this publication can be applied to each of them.

3 Problem Statement

First, we introduce basic terminology to describe data assignments.

Definition 3.1 (Storage Node). Data is stored on a set of storage nodes N . Each node $n \in N$ is associated with a storage capacity $n.capacity$.

Definition 3.2 (Data). The data to distribute is modeled as a set of data partitions P that may represent entire tables or (horizontal or vertical) table partitions. For each partition $p \in P$, we denote the associated size by $p.size$.

Definition 3.3 (Data Assignment). A data assignment $D : P \rightarrow \mathcal{P}(N)$ assigns each partition $p \in P$ to a subset of storage nodes ($\mathcal{P}(N)$ denotes the power set) with the semantics that copies of p are stored on nodes $D(p)$.

To be valid, data assignments must satisfy multiple types of constraints. Next, we define the constraints considered in our problem model.

Definition 3.4 (k -Safety). A data assignment satisfies the k -safety property iff for each partition p there are k nodes storing a copy, i.e., $\forall p \in P : |D(p)| = k$.

Definition 3.5 (Storage Constraints). A data assignment satisfies all storage constraints if, for each storage node, the sum of the sizes of all assigned partitions is at or below the storage threshold, i.e., $\forall n \in N : \sum_{p \in P: n \in D(p)} p.size \leq n.capacity$.

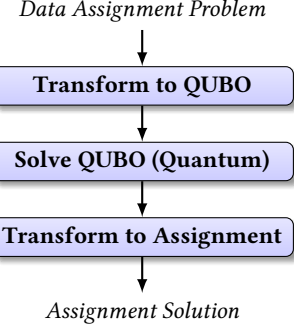


Figure 2: Overview of approach

Our goal is to optimize processing costs while satisfying all applicable constraints. Next, we introduce a simple cost model to model processing overheads.

Definition 3.6 (Processing Costs). We denote by r_{pn} the number of requests (per time unit) for partition $p \in P$ at node $n \in N$. Requests for data available at a node are served efficiently without network communication overheads. If the requested data is unavailable at a node, it must be sent over the network. We denote by c_p the cost of sending partition p over the network (assuming homogeneous connections, meaning that the cost does not depend on the source and target node). Hence, the total overhead for processing requests is $\sum_n \sum_{p \in U_n} r_{pn} \cdot c_p$ where $U_n \subseteq P$ denotes partitions unavailable at node $n \in N$.

Now, we are ready to introduce the problem that this paper focuses on.

Definition 3.7 (Assignment Problem). Given a set of storage nodes N , a set P of data partitions, a k -safety constraint, request frequencies r_{pn} , and communication costs c_p , find a data assignment D for P and N that satisfies k -safety and storage constraints while minimizing processing costs.

We map instances of the data assignment problem to quadratic unconstrained binary optimization (QUBO), the problem format supported by quantum annealer machines. Finally, we introduce this problem model formally.

Definition 3.8 (QUBO). An instance of quadratic unconstrained binary optimization (QUBO) is defined by a set V of binary variables and weights $w_{ij} \in \mathbb{R}$, connecting pairs of binary variables $v_i, v_j \in V$ with $i \leq j$. An optimal solution to a QUBO instance assigns variables to values such that $\sum_{v_i, v_j \in V: i \leq j} w_{ij}$ becomes minimal.

4 Overview

Figure 2 illustrates our approach to optimal data assignment. First, given an instance of the data assignment problem (see Definition 3.7), we transform the problem into an equivalent QUBO instance (see Definition 3.8). Next, we solve the QUBO instance using quantum computing. Quantum annealers can find optimal or near-optimal solutions to QUBO instances. Executing multiple annealing runs increases the chances of finding near-optimal solutions.

The optimal solution to the QUBO problem represents an optimal solution to the original assignment problem. Hence, we transform

Table 1: Symbols and their semantics.

Symbol	Semantics
N	set of storage nodes
$n.capacity$	storage capacity of node n
P	set of data partitions to distribute
$p.size$	storage size of partition p
k	store each partition on at least k nodes
r_{pn}	request count for partition p at node n
c_p	cost of sending partition p over the network
A_{pn}	1 iff partition p assigned to node n
I_n	storage chunks for node n
S_{in}	1 iff storage chunk i used on node n

the best QUBO solution found back into an equivalent solution to the data assignment problem. In the following two sections, we first describe our QUBO transformation in detail. Next, we analyze its formal properties.

5 QUBO Transformation

We describe a transformation from data assignment to QUBO.

5.1 Overview

Our QUBO formula has three components. First, Q_R reaches a minimal value of zero if data copies are stored redundantly on k nodes (i.e., the k -safety constraint is satisfied). Second, Q_S reaches a minimal value of zero if the combined sizes of all data assigned to each node do not exceed that node's capacity (i.e., the storage constraint is satisfied). Third, Q_P is proportional to the processing costs (lower values are preferable). The final QUBO formula sums up all three components:

$$Q = h \cdot (Q_R + Q_S) + Q_C \quad (1)$$

In this formula, h represents a weight high enough to prioritize satisfying constraints (components Q_R and Q_S) over minimizing costs (component Q_C). Without that weight, a solution that assigns data exceeding node capacity to lower processing costs may minimize the QUBO formula (despite being invalid). Both Q_R and Q_S have a value of at least one if constraints are violated. Processing costs are maximized if each request has to be answered by sending data from remote nodes:

$$\sum_{n \in N} \sum_{p \in P} (r_{pn} \cdot c_p) \quad (2)$$

At the same time, Equation 2 represents an upper bound on the amount of processing overheads saved by an assignment that violates constraints (compared to the best assignment satisfying all constraints). Hence, choosing $h > \sum_{n \in N} \sum_{p \in P} (r_{pn} \cdot c_p)$ ensures that a solution satisfying all constraints minimizes the QUBO formula. Next, we discuss the three components in more detail.

5.2 Safety Constraints

We introduce binary variables A_{pn} to describe data assignments. It is $A_{pn} = 1$ iff partition $p \in P$ is stored on node $n \in N$. Hence, for

each partition $p \in P$, the number of copies on all nodes is given as $\sum_{n \in N} A_{pn}$. We must ensure that the number of copies is k for each partition. The term $(\sum_{n \in N} A_{pn} - k)^2$ takes a minimal value of zero iff the number of copies is exactly equal to k . Hence, the following formula for Q_R takes minimal values if k copies are available for each partition:

$$Q_R = \sum_{p \in P} ((\sum_{n \in N} A_{pn}) - k)^2 \quad (3)$$

5.3 Storage Constraints

For each node $n \in N$, we divide its storage capacity $n.capacity$ into chunks $I_n \subseteq \mathbb{N}$ that sum up to the total storage capacity ($\sum_{i \in I_n} i = n.capacity$), and any value below the threshold can be represented as the sum of a subset of chunks. To simplify the notation, we assume that capacity values are powers of two such that $I_n = 2^{j_{0 \leq j \leq J_n}}$ satisfies both of the aforementioned requirements for a suitable choice of $J_n \in \mathbb{N}$ for any node $n \in N$.

We introduce binary variables S_{in} , indicating whether or not storage chunk $i \in I_n$ is used on node n . For any node n , we can represent any value from zero up to $n.capacity$ by the formula $\sum_{i \in I_n} i \cdot S_{in}$, using a suitable choice for the values of variables S_{in} . Therefore, denoting by $\sum_{p \in P} (A_{pn} \cdot p.size)$ the storage consumed by all partitions assigned to node n , the term $(\sum_{p \in P} (A_{pn} \cdot p.size) - \sum_{i \in I_n} i \cdot S_{in})^2$ can reach its minimal value of zero for suitable choices values of S_{in} , iff the total storage consumption does not exceed that node's capacity. Summing over all nodes yields a formula that can take a minimal value of zero iff this holds for each node:

$$Q_S = \sum_{n \in N} ((\sum_{p \in P} A_{pn} \cdot p.size - \sum_{i \in I_n} i \cdot S_{in})^2) \quad (4)$$

5.4 Processing Costs

We incur processing costs once data is requested at a node where no copy is readily available. The term $(1 - A_{pn})$ is equal to one iff the partition p is not available on node n . Hence, the term $r_{pn} \cdot c_p \cdot (1 - A_{pn})$ expresses the costs for processing requests for partition p on node n . To obtain overall processing costs, we sum over all partitions and nodes:

$$Q_C = \sum_{p \in P} c_p (\sum_{n \in N} r_{pn} \cdot (1 - A_{pn})) \quad (5)$$

5.5 Transformation Example

We illustrate the QUBO transformation on an example problem. We assume two data partitions, i.e., $P = \{p_1, p_2\}$, and two nodes: $N = \{n_1, n_2\}$. It is $p_1.size = 4$ and $p_2.size = 2$. The storage capacity of the two nodes is $n_1.capacity = 7$ and $n_2.capacity = 3$. We assume that $k = 1$, i.e., each partition only needs to be stored on one node. Also, we assume that the cost for sending data partitions over the network is proportional to their size (i.e., $c_1 = p_1.size = 4$ and $c_2 = p_2.size = 2$) and that the request frequency for both partitions is the same on each node and two times higher on the second node (i.e., $r_{11} = r_{21} = 1$ and $r_{12} = r_{22} = 2$). Clearly, an optimal assignment assigns as many partitions as possible to node n_2 , due to the higher frequency of requests. However, due to the storage

constraint, only the second partition (p_2) can be stored on node n_2 . Hence, storing the second partition on the second node while storing the first partition on the first node minimizes costs and satisfies all constraints (i.e., no node exceeds the storage capacity and the 1-safety constraint is satisfied since each copy is stored once).

For the QUBO problem, we introduce four variables to describe assignments, i.e., A_{pn} , where $p \in \{1, 2\}$ represents the two possible partitions and $n \in \{1, 2\}$ represents the two possible nodes. Also, we introduce variables representing storage chunks. The number of corresponding variables differs between the two nodes since their storage capacity differs. For the first node, introducing variables S_{i1} for $i \in \{1, 2, 4\}$ ensures that each possible storage consumption value between 0 and 7 can be represented. For the second node, introducing variables S_{i2} for $i \in \{1, 2\}$ ensures that each possible storage consumption value between 0 and 3 (the maximal storage capacity on that node) can be represented. Next, we construct a QUBO formula that uses these variables.

We need to represent safety constraints for both partitions. The following formula takes values higher than one if one of the two partitions is not stored on any of the two nodes (i.e., the 1-safety constraint is not satisfied).

$$Q_R = \sum_{p \in \{1,2\}} (A_{p1} + A_{p2} - 1)^2 \quad (6)$$

Next, we express storage constraints. The following QUBO formula takes values higher than one if the combined size of all partitions assigned to any of the two nodes exceeds that node's storage capacity.

$$Q_S = (4 \cdot A_{11} + 2 \cdot A_{21} - 1 \cdot S_{11} - 2 \cdot S_{21} - 4 \cdot S_{41})^2 + (4 \cdot A_{12} + 2 \cdot A_{22} - 1 \cdot S_{12} - 2 \cdot S_{22})^2 \quad (7)$$

Among all solutions satisfying constraints, we want to minimize processing costs. The following formula expresses the processing overheads, taking into account the cost of sending remote data over the network and the frequency at which specific partitions are requested.

$$Q_C = 1 \cdot 2 \cdot (1 - A_{11}) + 1 \cdot 4 \cdot (1 - A_{21}) + 2 \cdot 2 \cdot (1 - A_{12}) + 2 \cdot 4 \cdot (1 - A_{22}) \quad (8)$$

Finally, we multiply all QUBO components that relate to constraints (i.e., Q_R and Q_S) by a weight sufficiently high to ensure that the optimal QUBO solution sets all constraint-related components to zero. As discussed previously, setting the weight higher than the maximal processing costs for remotely accessing all partitions from all nodes satisfies this requirement:

$$h > 1 \cdot 2 + 1 \cdot 4 + 2 \cdot 2 + 2 \cdot 4 = 18 \quad (9)$$

In summary, the final QUBO formula is given as follows:

$$19 \cdot (Q_R + Q_S) + Q_C \quad (10)$$

In the optimal solution, the first partition is assigned to the second node while the second partition is assigned to the first node. Hence, we set $A_{11} = 0$ and $A_{21} = 1$ (representing assignments on the

first node) and set $A_{12} = 1$ and $A_{22} = 0$ (representing assignments on the second node). The total storage consumption on the first node is 4. Therefore, we set $S_{41} = 1$ but $S_{11} = S_{21} = 0$. The storage consumption on the second node is 2. Hence, we set $S_{22} = 1$ but $S_{12} = 0$.

6 Formal Analysis

First, we analyze the hardness of the data assignment problem.

THEOREM 6.1 (HARDNESS OF DATA ASSIGNMENT PROBLEM). *The data assignment problem is NP-hard.*

PROOF. We reduce instances of the knapsack problem to data assignment as follows. Given a set of items T , denote by $w_t \in \mathbb{R}$ the weight of item $t \in T$ and by u_t its utility. An optimal solution to the knapsack problem is a set $T^* \subseteq T$ that maximizes $\sum_{t \in T^*} u_t$ while not exceeding a given weight bound b : $\sum_{t \in T^*} w_t \leq b$. To transform a knapsack instance to a data assignment problem, we introduce one data partition p per item t , setting the size proportional to the weight (i.e., $w_t = p.size$). We set the costs for retrieving partition p , associated with item t , proportional to that item's utility (i.e., $c_p = u_t$). Also, we introduce two nodes n_1 and n_2 . For node n_1 , we set a capacity proportional to the weight threshold (i.e., $n_1.capacity = b$). For node n_2 , we set a capacity threshold that is high enough to store all data (i.e., the sum of all partition sizes). For node n_1 , we set the request frequency for each partition to the same unit value. For node n_2 , we set the request frequency for every partition to zero. Hence, only storing data on node n_1 reduces processing costs. We set the number of copies to one (i.e., $k = 1$). The optimal subset of partitions assigned to node n_1 , minimizing processing costs, corresponds to the optimal subset of items (maximizing utility). $\square$

This result justifies the use of sophisticated hardware for optimization. Next, we analyze the complexity of the problem in terms of the required number of qubits. This metric is typically the limiting factor when mapping problems to quantum annealers. First, we analyze how the asymptotic number of QUBO variables grows as a function of the input problem dimensions. We denote the number of nodes by $v = |N|$, the number of data partitions by $\pi = |P|$, and the maximal storage capacity by μ . We assume that storage capacity is always a power of two, meaning that we can use chunks that are powers of two as well (as discussed in the previous section).

THEOREM 6.2 (NUMBER OF VARIABLES). *The number of QUBO problem variables is in $O(v \cdot \pi + v \cdot \log(\mu))$.*

PROOF. We introduce one variable A_{pn} for each partition p and node n . Hence, the number of such variables is $v \cdot \pi$. Also, we introduce one variable S_{in} for each node n and storage chunk i . The number of storage chunks required to represent capacity up to μ is in $O(\log(\mu))$. Hence, the total number of such variables is in $O(v \cdot \log(\mu))$. $\square$

The number of problem variables provides a lower bound on the number of required qubits. However, quadratic terms in the QUBO formula must be represented by physical connections in the annealer hardware. In a fully connected architecture, quadratic terms between arbitrary qubits can be represented. In that case, it is sufficient to represent each QUBO variable by a single qubit.

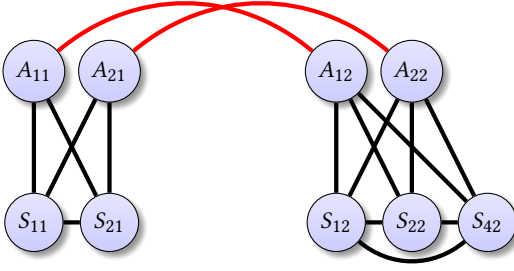


Figure 3: Illustrating the connection structure between problem variables of the example from Section 5.5. Black connections are due to storage constraints. Red connections are due to k -safety constraints.

COROLLARY 6.3 (NUMBER OF QUBITS: DENSE TOPOLOGIES). *In a fully connected topology, the number of required qubits is proportional to the number of problem variables: $O(v \cdot \pi + v \cdot \log(\mu))$.*

However, on typical annealer architectures (e.g., the Chimera topology, used by multiple D-Wave annealer models), each qubit is only connected to a constant number of other qubits. Hence, the number of quadratic terms in the QUBO formula with non-zero weights represents another (tighter) lower bound on the asymptotic number of required qubits. Next, we determine the asymptotic number of physical connections required between qubits.

THEOREM 6.4 (NUMBER OF CONNECTIONS). *The number of physical connections required between qubits is in $O(\pi \cdot v^2 + (\pi + \log(\mu))^2)$.*

PROOF. To represent k -safety constraints, for each partition $p \in P$, we need to represent quadratic terms connecting every pair of assignment variables A_{pn} associated with different nodes. Hence, the number of such connections is in $O(\pi \cdot v^2)$. To represent storage constraints, for each node $n \in N$, we need to represent quadratic terms connecting every pair of an assignment variable A_{pn} and storage chunk variable S_{in} . The number of such variables per node is $\pi + \log(\mu)$. Hence, the number of required connections is in $O((\pi + \log(\mu))^2)$. The formula representing processing costs does not contain any quadratic terms. $\square$

Example 6.5. Figure 3 illustrates required connections between the variables of the example problem, introduced in Section 5.5. Connections are required whenever two variables appear together in a quadratic term. The only components of the QUBO formula containing quadratic terms are the components representing k -safety constraints and storage constraints. The figure illustrates connections between problem variables, marking connections due to storage constraints in black and connections due to k -safety constraints in red.

As the number of physical connections per qubit is limited, multiple qubits must represent the same logical variable. For a constant number of connections per qubit, the asymptotic number of qubits equals the asymptotic number of required connections.

COROLLARY 6.6 (NUMBER OF QUBITS: SPARSE TOPOLOGIES). *If each qubit is connected to a constant number of other qubits, the*

required number of qubits is lower-bounded by the number of required connections: $O(\pi \cdot v^2 + (\pi + \log(\mu))^2)$.

Note that this is only a lower bound. Depending on the qubit topology, the number of required qubits may grow superlinearly in the number of required connections.

7 Related Work

This paper is part of a line of work that studies applications of quantum computers to database systems. This line of work has started with Grover’s algorithm for finding items in unsorted sets [15, 17]. Follow-up work has explored implementations of set and relational operators that are based on quantum computation [9, 19, 20, 26, 27, 33, 36, 51].

Our work relates most to a line of prior work that uses quantum computing to solve hard optimization problems that arise in the context of database systems, starting with initial work on multiple query optimization [44]. More recent work [10, 31, 38–40, 47] expands the scope to other query optimization variants and different types of hardware. Other prior work uses quantum computing to solve problems such as index selection [18, 25, 45], transaction scheduling [5, 6, 14], and schema matching [11]. This work differs from all prior work by its focus on data partitioning problems.

At the same time, this work relates to prior research on physical design optimization, including distributed data allocation [3, 12, 28] and data partitioning [2, 32, 35, 41–43] in particular, as well as index selection [4, 7, 29] and materialized view selection [8, 21, 46, 49]. Whereas prior work in those domains proposes algorithms that are executed on classical machines, this work differs by the use of quantum computing.

8 Conclusion and Outlook

The decision to allocate data to nodes has a significant impact on performance in a distributed database management system. This paper describes a first approach to map the data assignment problem to instances of QUBO, a problem format supported by current quantum annealers. Furthermore, we present theoretical results, analyzing how the number of qubits (one of the primary limiting factors on current annealer machines) grows as a function of the problem dimensions.

Going forward, we plan to address the following topics. First, we will complement our theoretical results with experiments. In those experiments, we will measure optimization overheads and result quality for representative test cases. Second, we will expand our formal model. In particular, besides requests for single data partitions, we will consider more complex queries that may connect multiple partitions and tables. In particular, this step will require extensions to the processing cost formulas to model the performance of different query processing strategies, enabled by specific data assignments. Third, acknowledging the limited number of qubits on current annealer machines, we plan to investigate domain-specific problem decomposition methods, dividing an instance of data assignment optimization into multiple smaller sub-problems. This extension increases the size of problem instances that can be solved using a given annealer machine.

References

- [1] Scott Aaronson. 2013. *Quantum Computing since Democritus*. Cambridge University Press. 1–398 pages. <https://doi.org/10.1017/CBO9780511979309>
- [2] Sanjay Agrawal, Vivek Narasayya, and Beverly Yang. 2004. Integrating vertical and horizontal partitioning into automated physical database design. In *SIGMOD*. 359–370. doi:10.1145/1007568.1007609
- [3] Peter M.G. Apers. 1988. Data Allocation in Distributed Database Systems. *ACM Transactions on Database Systems (TODS)* 13, 3 (1988), 263–304. doi:10.1145/44498.45063
- [4] Zohreh Asgharzadeh Talebi, Rada Chirkova, and Yahya Fathi. 2013. An integer programming approach for the view and index selection problem. *DKE* 83 (2013), 111–125. doi:10.1016/j.data.2012.11.001
- [5] Tim Bittner and Sven Groppe. 2020. Avoiding blocking by scheduling transactions using quantum annealing. In *Proceedings of the 24th Symposium on International Database Engineering and Applications*. 1–10. doi:10.1145/3410566.3410593
- [6] Tim Bittner and Sven Groppe. 2020. Hardware Accelerating the Optimization of Transaction Schedules via Quantum Annealing by Avoiding Blocking. *Open Journal of Cloud Computing* 7, 1 (2020), 1–21. <http://nbn-resolving.de/urn:nbn:de:101:1-2020112218332015343957>
- [7] Surajit Chaudhuri. 2004. Index selection for databases: A hardness study and a principled heuristic solution. *KDE* 16, 11 (2004), 1313–1323. doi:10.1109/TKDE.2004.75
- [8] Rada Chirkova, Alon Y. Halevy, and Dan Suciu. 2002. A formal perspective on the view selection problem. *Vldb Journal* 11, 3 (2002), 216–237. doi:10.1007/s00778-002-0070-0
- [9] Paul Cockshott. 1997. Quantum Relational Databases. *CoRR* abs/971202 (1997), 1–7. arXiv:9712025 [quant-ph] <http://arxiv.org/abs/quant-ph/9712025>
- [10] Tobias Fankhauser, Marc E. Soler, Rudolf M. Fuchslin, and Kurt Stockinger. 2021. Multiple Query Optimization using a Hybrid Approach of Classical and Quantum Computing. *CoRR* abs/2107.1 (2021), 1–18. arXiv:2107.10508 <http://arxiv.org/abs/2107.10508>
- [11] Kristin Fritsch and Stefanie Scherzinger. 2023. Solving Hard Variants of Database Schema Matching on Quantum Computers. In *PVLDB*, Vol. 16. 3990–3993. doi:10.14778/3611540.3611603
- [12] De Gope. 2012. Dynamic Data Allocation Methods in Distributed Database System. *American Academic and Scholarly Research* ... 4, 6 (2012). <http://naturalspublishing.com/files/published/15j7d2xw82j2v4.pdf>
- [13] Hayato Goto, Kosuke Tatsumura, and Alexander R. Dixon. 2019. Combinatorial optimization by simulating adiabatic bifurcations in nonlinear Hamiltonian systems. *Science Advances* 5, 4 (2019), 1–8. doi:10.1126/sciadv.aav2372
- [14] Sven Groppe and Jinghua Groppe. 2021. Optimizing Transaction Schedules on Universal Quantum Computers via Code Generation for Grover’s Search Algorithm. In *International Database Engineering and Applications Symposium*. 149–156. doi:10.1145/3472163.3472164
- [15] LK Grover. 1996. A fast quantum mechanical algorithm for database search. In *Symposium on the Theory of Computing*. 212–219. doi:10.1145/237814.237866
- [16] Lov K Grover. 1997. Quantum computers can search arbitrarily large databases by a single query. *Physical Review Letters* 79, 23 (1997), 4709–4712. doi:10.1103/PhysRevLett.79.4709
- [17] Lov K. Grover. 1997. Quantum mechanics helps in searching for a needle in a haystack. *Physical Review Letters* 79, 2 (1997), 325–328. doi:10.1103/PhysRevLett.79.325
- [18] Le Gruenwald, Tobias Winker, Umut Çalikylmaz, Jinghua Groppe, and Sven Groppe. 2023. Index Tuning with Machine Learning on Quantum Computers for Large-Scale Database Applications. In *CEUR Workshop Proceedings*, Vol. 3462. 1–9. <https://ceur-ws.org/Vol-3462/QDSM5.pdf>
- [19] Amor Gueddana, Rihab Chatta, and Nouredine Boudriga. 2010. Optimized methods for inserting and deleting records and data retrieving in quantum database. In *2010 12th International Conference on Transparent Optical Networks, ICTON 2010*. 1–5. doi:10.1109/ICTON.2010.5549171
- [20] Szabolcs Jöczik and Attila Kiss. 2020. Quantum Computation and Its Effects in Database Systems. In *Communications in Computer and Information Science*, Vol. 1259 CCIS. 13–23. doi:10.1007/978-3-030-54623-6_2
- [21] Shantanu Joshi and Christopher Jermaine. 2008. Materialized sample views for database approximation. *ICDE* 20, 3 (2008), 337–351. doi:10.1109/TKDE.2007.190664
- [22] Robert Kallman, Hideaki Kimura, Jonathan Natkins, Andrew Pavlo, Alexander Rasin, Stanley Zdonik, Evan P.C. Jones, Samuel Madden, Michael Stonebraker, Yang Zhang, John Hugg, and Daniel J. Abadi. 2008. H-Store: A high-performance, distributed main memory transaction processing system. *Proceedings of the VLDB Endowment* 1, 2 (2008), 1496–1499. doi:10.14778/1454159.1454211
- [23] Hiroshi Kanai, Masashi Yamashita, Kotaro Tanahashi, and Shu Tanaka. 2024. Annealing-Assisted Column Generation for Inequality-Constrained Combinatorial Optimization Problems. *IEEE Access* 12 (2024), 157669–157685. doi:10.1109/ACCESS.2024.3486768
- [24] Yu-Ting Kao, Jia-Le Liao, and Hsiu-Chuan Hsu. 2023. Solving Combinatorial Optimization Problems on Fujitsu Digital Annealer. *CoRR* abs/2311.0 (2023), 1–5. arXiv:2311.05196 <http://arxiv.org/abs/2311.05196>
- [25] Manish Kesarwani and Jayant R. Haritsa. 2024. Index Advisors on Quantum Platforms. *Proceedings of the VLDB Endowment* 17, 11 (2024), 3615–3628. doi:10.14778/3681954.3682025
- [26] Manish Kesarwani and Jayant R. Haritsa. 2024. Is Quantum-Based SQL Query Execution Viable?. In *QDSM*. 1–7. <https://vldb.org/workshops/2024/proceedings/QDSM/QDSM.8.pdf>
- [27] Yang Liu and Gui Lu Long. 2007. Deleting a marked item from an unsorted database with a single query. *CoRR* abs/0710.3 (2007), 1–4. <https://arxiv.org/abs/0710.3301>
- [28] Salvatore T. March and Sangkyu Rho. 1995. Allocating Data and Operations to Nodes in Distributed Database Design. *IEEE Transactions on Knowledge and Data Engineering* 7, 2 (1995), 305–317. doi:10.1109/69.382299
- [29] Frank Martin, Edward Omiecinski, and Shamkant Navathe. 1992. Adaptive and Automated Index Selection in RDBMS. In *EDBT*. 277–292.
- [30] Hiroshi Nakayama, Junpei Koyama, Noboru Yoneoka, and Toshiyuki Miyazawa. 2021. *Description: Third Generation Digital Annealer Technology*. Technical Report. 1–11 pages. https://www.fujitsu.com/global/documents/about/research/techintro/3rd-g-da_en.pdf
- [31] Nitin Nayak, Jan Rehfeld, Tobias Winker, Benjamin Warnke, Umut Çalikylmaz, and Sven Groppe. 2023. Constructing Optimal Bushy Join Trees by Solving QUBO Problems on Quantum Hardware and Simulators. In *Proceedings of the International Workshop on Big Data in Emergent Distributed Environments*. 1–7. doi:10.1145/3579142.3594298
- [32] Rimma Nehme and Nicolas Bruno. 2011. Automated partitioning design in parallel database systems. In *SIGMOD*. 1137–1148. doi:10.1145/1989323.1989444
- [33] Chao Yang Pang, Ri Gui Zhou, Cong Bao Ding, and Ben Qiong Hu. 2013. Quantum search algorithm for set operation. *Quantum Information Processing* 12, 1 (2013), 481–492. doi:10.1007/s11228-012-0385-8 arXiv:0806.1539
- [34] D. Pathania, S. Momose, T. Nishimura, and M. Ikuta. 2022. Accelerating and Optimizing Oil and Gas Exploration Planning Using Quantum Inspired Classical Computing or Vector Annealing. In *Sixth EAGE High Performance Computing Workshop*. 1–7. doi:10.3997/2214-4609.2022615018
- [35] Jun Rao, Chun Zhang, Nimrod Megiddo, and Guy Lohman. 2002. Automating physical database design in a parallel database. In *SIGMOD*. 558–569. doi:10.1145/564691.564757
- [36] Tamer Salman and Yoram Baram. 2012. Quantum set intersection and its application to associative memory. *Journal of Machine Learning Research* 13 (2012), 3177–3206.
- [37] Masataka Sao, Hiroyuki Watanabe, Yuuichi Musha, and Akihiro Utsunomiya. 2019. Application of digital annealer for faster combinatorial optimization. *Fujitsu Scientific and Technical Journal* 55, 2 (2019), 45–51.
- [38] Pranshi Saxena, Ibrahim Sabek, and Federico Spedalieri. 2024. Constrained Quadratic Model for Optimizing Join Orders. In *Q-Data*. 38–44. doi:10.1145/3665225.3665447
- [39] Manuel Schönberger. 2022. Applicability of Quantum Computing on Database Query Optimization. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 2512–2514. doi:10.1145/3514221.3520257
- [40] Manuel Schönberger, Stefanie Scherzinger, and Wolfgang Maurer. 2023. Ready to Leap (by Co-Design)? Join Order Optimisation on Quantum Hardware. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–27. doi:10.1145/3588946
- [41] Marco Serafini, Rebecca Taft, Aaron J. Elmore, Andrew Pavlo, Ashraf Aboulmaga, and Michael Stonebraker. 2016. Clay: Fine-Grained Adaptive Partitioning for General Database Schemas. *Vldb* 10, 4 (2016), 445–456. doi:10.14778/3025111.3025125
- [42] Liwen Sun, Michael J Franklin, Jiannan Wang, and Eugene Wu. 2016. Skipping-oriented Partitioning for Columnar Layouts. *PVLDB* 10, 4 (2016), 421–432. doi:10.14778/3025111.3025123
- [43] Khai Q Tran and Jeffrey F Naughton. 2014. JECB: a join-extension, code-based approach to OLTP data partitioning. In *SIGMOD*. 39–50. doi:10.1145/2588555.2610532
- [44] Immanuel Trummer and Christoph Koch. 2016. Multiple Query Optimization on the D-Wave 2X Adiabatic Quantum Computer. *PVLDB* 9, 9 (2016), 648–659. doi:10.14778/2947618.2947621
- [45] Immanuel Trummer and Davide Venturelli. 2024. Leveraging Quantum Computing for Database Index Selection. In *Q-Data*. 14–26. doi:10.1145/3665225.3665445
- [46] T. V. Vijay Kumar and Santosh Kumar. 2012. Materialized view selection using genetic algorithm. In *Communications in Computer and Information Science*, Vol. 306 CCIS. 225–237. doi:10.1007/978-3-642-32129-0_26
- [47] Tobias Winker, Umut Çalikylmaz, Le Gruenwald, and Sven Groppe. 2023. Quantum Machine Learning for Join Order Optimization using Variational Quantum Circuits. In *Proceedings of the International Workshop on Big Data in Emergent Distributed Environments*. 1–7. doi:10.1145/3579142.3594299
- [48] Masanao Yamaoka, Takuya Okuyama, Masato Hayashi, Chihiro Yoshimura, and Takashi Takemoto. 2019. CMOS Annealing Machine: An In-memory Computing Accelerator to Process Combinatorial Optimization Problems. In *Proceedings of the Custom Integrated Circuits Conference*. 1–8. doi:10.1109/CICC.2019.8780296
- [49] Jian Yang, Kamalakar Karlapalem, and Qing Li. 1997. Algorithms for materialized view design in data warehousing environment. In *VLDB*. 136–145. <http://www.>

- vldb.org/conf/1997/P136.PDF
- [50] Andrew Chi Chih Yao. 1993. Quantum Circuit Complexity. In *Annual Symposium on Foundations of Computer Science (Proceedings)*. 352–361. doi:10.1109/sfcs.1993.366852
- [51] Ahmed Younes. 2007. Database manipulation on quantum computers. *CoRR* abs/0705.4 (2007), 1–15. arXiv:arXiv:0705.4303v1 <http://arxiv.org/abs/0705.4303>